An Image Captioning System with Attention

A CNN Encoder, Attention Mechanism, and LSTM Decoder

Task Definition

- The objective is to generate a natural-language caption for a given input image.
- Two properties make this task non-trivial:
  - The input (pixels) and output (words) belong to different modalities.
  - Multiple valid captions can describe the same image.
- The system is expected to learn general descriptive patterns rather than memorize fixed sentences.

System Overview

Three components form the architecture:

1. **CNN Encoder** – extracts visual features from the image.
2. **Attention Mechanism** – determines which image regions are relevant at each generation step.
3. **LSTM Decoder** – generates the caption sequentially, one word at a time.

This design was selected over a non-attentive encoder-decoder baseline, as it allows the intermediate attention weights to be inspected and visualized.

Encoder

- ResNet-50, pretrained on ImageNet, with the final pooling and classification layers removed.
- Adaptive average pooling produces a fixed 5×5 spatial grid (25 regions), each represented by a 2048-dimensional feature vector.
- Encoder weights are frozen for the majority of training.
- The final convolutional block is unfrozen at epoch 21, allowing limited fine-tuning once the decoder has stabilized.

Attention Mechanism

At decoding step t, an energy score is computed for each region a_i using the decoder's hidden state h_{t-1}:

$$e_{t,i} = w^\top \tanh\left(W_a\,[a_i\,;\,h_{t-1}]\right)$$

Normalized via softmax across all regions:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{25}\exp(e_{t,j})}$$

The context vector is the attention-weighted sum of region features:

$$z_t = \sum_{i=1}^{25} \alpha_{t,i}\,a_i$$

A Bahdanau-style (concatenation-based) formulation was adopted for its interpretability and suitability for a moderate region count.

Decoder

- Implemented as an `LSTMCell`, executed step-by-step to allow attention context injection at every step.
- Hidden state initialized from the mean of all encoded regions, providing global image context prior to attention.

$$(h_t, c_t) = \text{LSTMCell}\big([\,E(y_{t-1})\,;\,z_t\,],\ (h_{t-1}, c_{t-1})\big)$$

$$p(y_t \mid y_{<t}, \text{image}) = \text{softmax}\big(W_o \cdot \text{Dropout}(\text{LayerNorm}(h_t))\big)$$

Generation proceeds word-by-word from <SOS> until <EOS> or a maximum length is reached.

Training Objective

Cross-entropy loss, combined with an attention regularization term:

$$\mathcal{L} = \underbrace{-\sum_{t} \log p(y_t \mid y_{<t}, \text{image})}_{\text{cross-entropy}} + \lambda \sum_{i=1}^{25} \left(1 - \sum_{t} \alpha_{t,i}\right)^2$$

- The regularization term discourages the model from ignoring image regions or repeatedly attending to a single region.

Scheduled Sampling

$$p_{\text{tf}}(\text{epoch}) = 1.0 + (0.7 - 1.0) \cdot \frac{\text{epoch}}{\text{num\_epochs} - 1}$$

- Teacher forcing is decayed linearly from 100% to 70% across training.
- This exposes the decoder increasingly to its own predictions, matching inference-time conditions where ground-truth tokens are unavailable.

Optimization and Inference-Time Decoding

Optimization:

- Adam optimizer, learning rate 3×10^{-4}.
- Gradient clipping at norm 5.0.
- Early stopping (patience = 5 epochs), based on validation loss.
- BLEU-4 evaluated every 10 epochs as a secondary signal.

Decoding:

- Greedy decoding: fastest, cannot recover from early errors.
- Beam search (width = 3): retains top candidate sequences; used as the default for final demonstrations.

Stage One: Initial Implementation

- The pipeline was first implemented as five separate Python modules (vocabulary/dataset, model, training loop, utilities, inference), enabling isolated testing of each component.
- Initial configuration:
  - Encoder: ResNet-101
  - Attention grid: 8×8 (64 regions)
- This configuration was expected to capture finer visual detail, but resulted in substantially longer training times given the available dataset size and hardware.

Stage Two: Optimization for Training Time

Component	Change	Rationale
Encoder	ResNet-101 $\rightarrow$ ResNet-50	Faster, comparable feature quality
Attention grid	$8 \times 8 \rightarrow 5 \times 5$	Reduces per-step attention cost
Embed./hidden size	Kept at 128 / 256	Avoids unnecessary decoder capacity
DataLoader workers	$6 \rightarrow 0$	Avoided multiprocessing hang
Frozen encoder	Autograd graph skipped	Saves memory/compute while frozen
BLEU frequency	Every 5 $\rightarrow$ every 10 epochs	Reduces evaluation overhead

These changes were made to reduce training time within the available hardware, not to alter model capability.

Data Leakage in the Original Demonstration Pipeline

- The initial demonstration script sampled 800 images at random from the full 8,000-image dataset, rather than from the held-out test set.
- Given the train/validation/test split (6,473 / 809 / 809 images), approximately 81% of demonstration images had already been seen during training.
- **Correction:** the exact train/validation/test split was reconstructed deterministically (same filenames, same random seed), and demonstration images were restricted to the reconstructed test set only, with a verification step applied before caption generation.

Inference Pipeline Enhancements

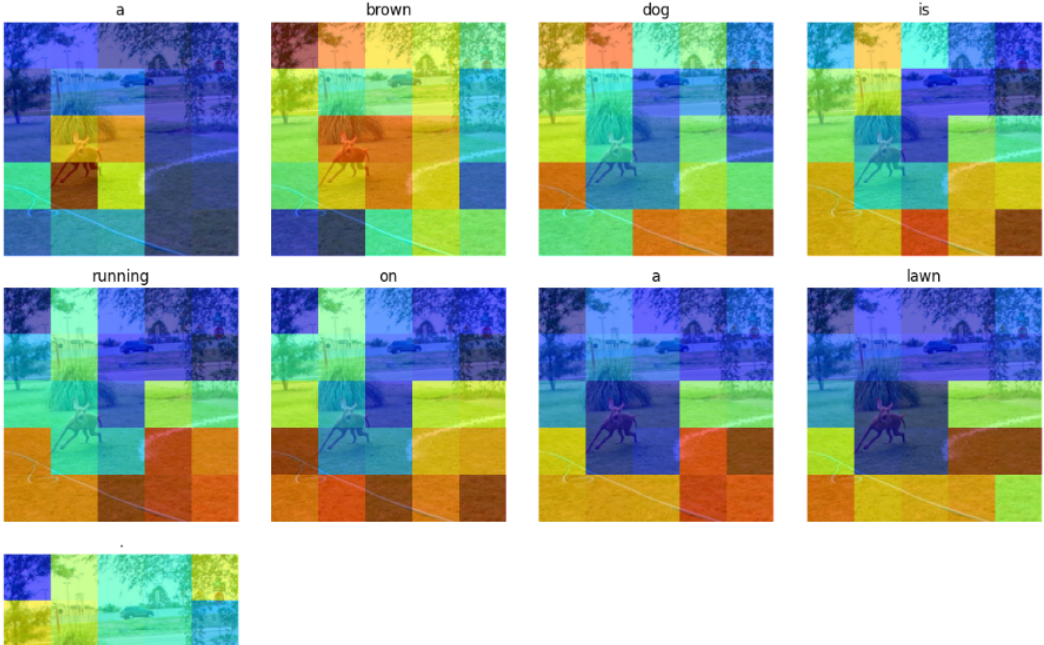
- Automatic detection of the most recent model checkpoint.
- Checkpoint file-size validation, to detect incomplete/corrupted saves before loading.
- Capped live preview to avoid the appearance of a stalled notebook during batch processing.
- Attention-weight visualization (heatmap overlays) for qualitative analysis.
- Batch inference producing a structured CSV of predictions and contact-sheet image grids for rapid visual review.

Training and Validation Loss

Epoch	Train Loss	Val Loss	Note
1	4.0650	6.4070	first epoch
10	2.5153	5.8667	BLEU-4 = 0.0633
20	2.4302	5.4883	BLEU-4 = 0.0579
21	2.7121	5.2307	encoder unfrozen
26	2.5506	5.0964	best model
30	2.5009	5.1952	BLEU-4 = 0.0619

- Both losses decrease steadily; a brief increase follows encoder unfreezing at epoch 21.
- BLEU-4 is interpreted as a relative progress signal, computed against a single reference caption per image.

Caption: a brown dog is running on a lawn .

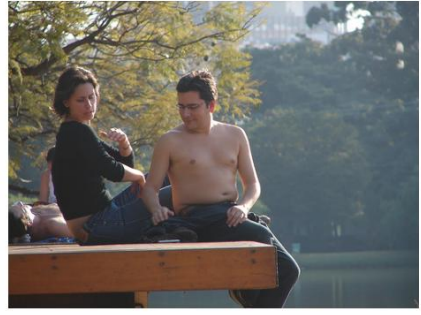


Qualitative Results: Successful Captioning

a brown dog is running on a lawn .

"A brown dog chases the water from a sprinkler on a lawn."

a man and a woman sitting on a dock .



"A lady and a man with no shirt sit on a dock."

Failure Mode: Grammatically Incorrect Caption

a man is drilling a ice drilling a lake .

"A man drilling a hole in the ice."

a man in a brown dog is jumping in the air .

"A dog jumping in the air to retrieve a stick from a man."

Failure Mode: Grammatically Correct but Semantically Incorrect

a brown dog is jumping in the air .

“,A brown dog is sitting in some long grass.”

a man is holding a paper .

“A man sits at a table in front of a computer wearing headphones.”

Failure Mode: Incomplete Caption

a young girl in a blue shirt and a <UNK> <UNK> .

"People walk by a display of underwear hanging in a stall."

a man in a harness is being <UNK> in the water .

"A man in a harness being dragged across the water."

Conclusions

- A functional image captioning system was implemented using a ResNet-50 encoder, attention mechanism, and LSTM decoder.
- Dataset splitting by image, rather than by caption, was required to prevent evaluation on near-duplicate training samples.
- Model configuration (encoder depth, attention grid size) was adjusted to balance training time against available hardware.
- Scheduled sampling, gradient clipping, and attention regularization contributed to training stability.
- A data leakage issue in the demonstration pipeline was identified and corrected prior to final evaluation.
- Observed failure modes (repetition, semantic errors, vocabulary gaps) suggest that a larger dataset, expanded vocabulary, and extended training would be the most direct paths to improvement.

Thank You